DESIGN AND ANALYSIS OF ALGORITHMS

Assignment 2 (CO2)

Brute-Force and Exhaustive-Search Techniques: Prediction, Measurement and Redesign

Submitted by	N D YOGESHWAR
Registration number	192465067
Random seed for all test data	5067 (last four digits of the registration number)
Programming language	Python 3.11
GitHub repository	https://github.com/yogeshwardev/DAA-A2-192465067-N-D-YOGESHWAR

Abstract

This report investigates six brute-force and exhaustive-search problems and then redesigns the one that scales worst. Selection, bubble and insertion sort are compared on six input arrangements; sequential search is studied for different positions, list sizes and loads; brute-force string matching is tested on inputs designed to cause little and much work; closest pair and convex hull are derived from geometric properties; and an exhaustive search for the best airlift load is analysed up to the point where it stops being usable. For every question a prediction was written first, operation counts were recorded as the main evidence, and the observed behaviour was compared with the prediction.

The exhaustive search of Q6 was chosen for the design challenge. Its bottleneck is that it enumerates all 2^n subsets and recomputes every sum from scratch. A depth-first search that prunes overweight branches and uses a value-density bound was designed, predicted, implemented and measured, and it examined a small fraction of the original candidates while giving the same optimum. The report also records the instances on which this redesign loses most of its advantage.

Contents

Q1. Sorting under different data conditions

Q2. Sequential search as an engineering choice

Q3. Behaviour of brute-force string matching

Q4. Closest pair

Q5. Convex hull

Q6. Exhaustive search and combinatorial growth

Q7. Day-4 design challenge: diagnose, redesign and validate

Conclusion and references

Note on measurements. Operation counts are the primary evidence in this report. Execution times were measured on one laptop with Python and vary from run to run, so they are used only as supporting evidence.

Q1. Sorting under Different Data Conditions

1.1 Predictions made before implementation

The three algorithms differ in how much of their work depends on the order of the input. Selection sort always scans the whole unsorted part to find the minimum, so its number of comparisons is n(n - 1)/2 for every input, and it makes at most n - 1 swaps. Bubble sort with an early-exit flag stops after a pass without swaps, so it needs n - 1 comparisons on sorted data, but on other data it needs as many passes as the farthest displacement, and it performs one swap for every inversion. Insertion sort moves an element left past larger elements only, so the number of element moves equals the number of inversions and the comparisons are about n - 1 plus the number of inversions.

From this the expected results for n = 2000 are: sorted input, 1,999 comparisons and no moves for bubble and insertion sort and 1,999,000 comparisons for selection sort; reversed input, 1,999,000 comparisons for all three and about 2 million swaps or moves for bubble and insertion sort; random input, about half as many moves ($n^2/4$, since a random permutation has about n(n - 1)/4 inversions).

1.2 Test data

Six datasets of 2,000 priority scores were generated with seed 5067. Two contrasting datasets are the random order (many inversions) and the already sorted order (none). The reversed order is the most disordered case. Because the fulfilment centre reports that 95 percent of orders are already in order, two datasets resemble that situation: 1,900 sorted scores followed by 100 new random ones, and 1,980 sorted scores followed by a small batch of 20 new ones. The edge case is a list that contains only five distinct scores, which produces many equal keys.

1.3 Results

Dataset (n = 2000)	**Selection sort comp / swaps**	**Bubble sort comp / swaps**	**Insertion sort comp / moves**
random	1,999,000 / 1,991	1,998,675 / 1,003,185	1,005,176 / 1,003,185
sorted	1,999,000 / 0	1,999 / 0	1,999 / 0
reversed	1,999,000 / 1,000	1,999,000 / 1,999,000	1,999,000 / 1,999,000
nearly sorted 95%	1,999,000 / 1,989	1,998,439 / 100,462	102,461 / 100,462
sorted + batch of 20	1,999,000 / 1,981	1,998,829 / 23,604	25,603 / 23,604
few distinct scores	1,999,000 / 1,584	1,919,200 / 815,745	817,742 / 815,745

Table 1.1: Comparisons and swaps or moves (from results/q1.csv)

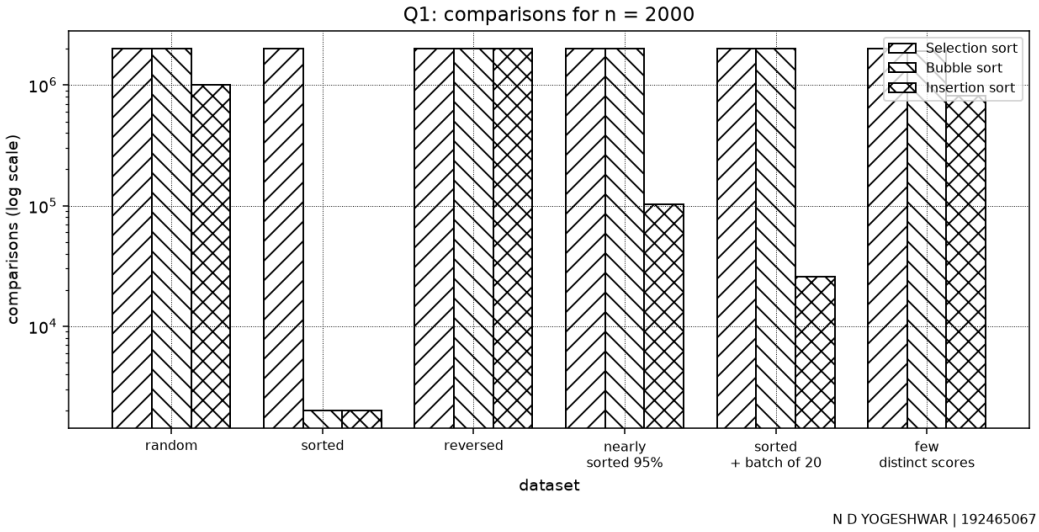


Figure 1.1: Comparisons for the six datasets (logarithmic axis)

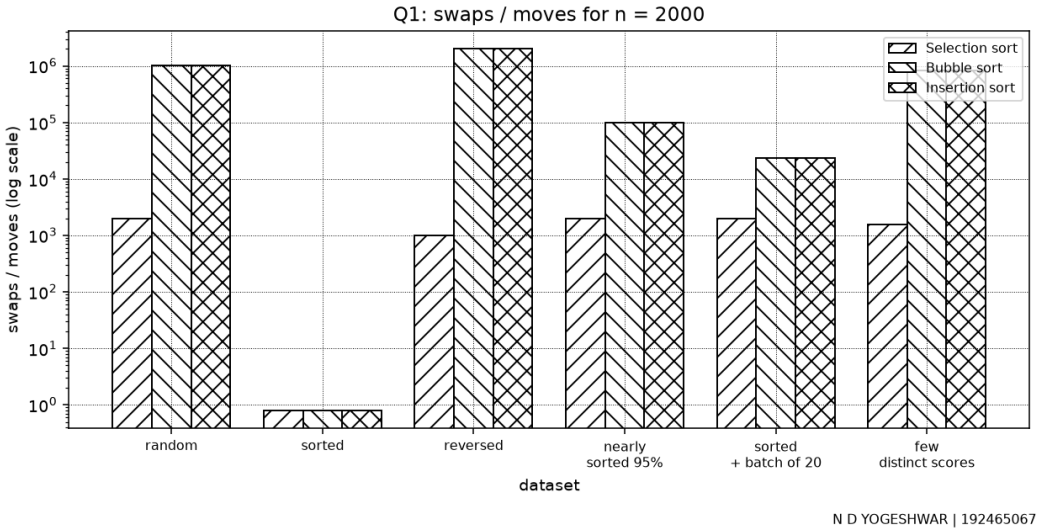


Figure 1.2: Swaps or moves for the six datasets (logarithmic axis)

```
C:\DAA-A2\src>  python q1_sorting.py

======================================================================
Date/Time: 12-10-2026 22:27:27
Q1 - SELECTION, BUBBLE AND INSERTION SORT
======================================================================
n = 2000, seed = 5067

PREDICTIONS (made before running), comparisons / swaps-or-moves:
  sorted               Selection (1999000, 0); Bubble (1999, 0); Insertion (1999, 0)
  reversed             Selection (1999000, 1000); Bubble (1999000, 1999000); Insertion (1999000, 1999000)
  random               Selection (1999000, 1999); Bubble (1999000, 999500); Insertion (999500, 999500)
  nearly sorted 95%    selection unchanged; bubble needs passes until the last displaced item is placed;
                       insertion should do far fewer moves than on random data

dataset               algorithm          comparisons   swaps/moves   time(ms)   sorted
random                Selection sort         1999000          1991     149.06     True
random                Bubble sort            1998675       1003185     370.01     True
random                Insertion sort         1005176       1003185     200.90     True
sorted                Selection sort         1999000             0     132.05     True
sorted                Bubble sort               1999             0       0.17     True
sorted                Insertion sort            1999             0       0.26     True
reversed              Selection sort         1999000          1000     200.14     True
reversed              Bubble sort            1999000       1999000     570.63     True
reversed              Insertion sort         1999000       1999000     432.49     True
nearly sorted 95%     Selection sort         1999000          1989     235.80     True
nearly sorted 95%     Bubble sort            1998439        100462     248.18     True
nearly sorted 95%     Insertion sort          102461        100462      17.37     True
sorted + batch of 20  Selection sort         1999000          1981     199.59     True
sorted + batch of 20  Bubble sort            1998829         23604     253.06     True
sorted + batch of 20  Insertion sort           25603         23604       5.60     True
few distinct scores   Selection sort         1999000          1584     256.80     True
few distinct scores   Bubble sort            1919200        815745     377.58     True
few distinct scores   Insertion sort          817742        815745     160.75     True

GROWTH WITH n (nearly sorted 95% and random):
dataset                     n  algorithm        comparisons   swaps/moves
nearly sorted 95%         500  Selection sort        124750           472
nearly sorted 95%         500  Bubble sort           123970          6394
nearly sorted 95%         500  Insertion sort          6893          6394
nearly sorted 95%        1000  Selection sort        499500           977
nearly sorted 95%        1000  Bubble sort           498939         20940
nearly sorted 95%        1000  Insertion sort         21939         20940
nearly sorted 95%        2000  Selection sort       1999000          1992
nearly sorted 95%        2000  Bubble sort          1995259         85370
nearly sorted 95%        2000  Insertion sort         87369         85370
nearly sorted 95%        4000  Selection sort       7998000          3969
nearly sorted 95%        4000  Bubble sort          7996347        375013
nearly sorted 95%        4000  Insertion sort        379012        375013
random                    500  Selection sort        124750           489
random                    500  Bubble sort           124344         61223
random                    500  Insertion sort         61717         61223
random                   1000  Selection sort        499500           987
random                   1000  Bubble sort           499490        261091
random                   1000  Insertion sort        262082        261091
random                   2000  Selection sort       1999000          1997
random                   2000  Bubble sort          1997965        992201
random                   2000  Insertion sort        994193        992201
random                   4000  Selection sort       7998000          3990
random                   4000  Bubble sort          7990619       4028152
random                   4000  Insertion sort       4032144       4028152

                                                  N D YOGESHWAR | Reg No 192465067
```

Figure 1.3: Program output for Q1

1.4 Prediction compared with observation

The predictions for sorted and reversed input were confirmed exactly: selection sort made 1,999,000 comparisons on every dataset, and bubble and insertion sort made 1,999 comparisons and no swaps on sorted input. On random input the moves of insertion sort were 1,003,185, close to

the predicted n(n - 1)/4 = 999,500, which confirms the inversion argument.

One prediction was only partly right. I expected bubble sort to profit from its early exit on the 95 percent sorted data, but it still made 1,998,439 comparisons, almost the same as on random data. The reason is that in a pass a large element can travel all the way to the right, whereas a small element moves only one place to the left. New orders with small priority scores appended at the end therefore need a pass for every position they must travel, so the number of passes is close to n and the early exit never helps. Insertion sort does not have this problem, because it places each element directly. On the same data it made only 102,461 comparisons, and for the batch of 20 new orders only 25,603.

Selection sort is unaffected by the order of the data in its comparisons, which is both its weakness (it cannot benefit from existing order) and its strength (its swaps stay below n - 1, which would matter if writing an element were very expensive). With few distinct scores, bubble and insertion sort both needed fewer moves than on random data, because equal elements are not swapped.

1.5 Design decision

Insertion sort is recommended for the fulfilment centre. When about 95 percent of the orders are already in order, it needed 102,461 comparisons against about two million for selection and bubble sort, roughly 20 times fewer, and 25,603 comparisons when only a small batch of new orders is added, which is about 78 times fewer. Its cost is proportional to the number of inversions, which is exactly the quantity that is small for nearly sorted data. The growth experiment in Table 1.2 shows that the benefit is not an accident of one size.

n	Insertion, nearly sorted 95% comparisons	Insertion, random comparisons	Bubble, nearly sorted 95% comparisons
500	6,893	61,717	123,970
1000	21,939	262,082	498,939
2000	102,461	1,005,176	1,998,439
4000	379,012	4,032,144	7,996,347

Table 1.2: Growth with n

A limitation should be stated. If the 5 percent of new orders is large and scattered, insertion sort still has quadratic cost (it was about ten times cheaper than on random data, not linear). A better practical design for periodic batches would be to sort the small batch alone and merge it with the existing sorted list, but this is outside the three algorithms compared in this question.

Q2. Sequential Search as an Engineering Choice

2.1 Experiment design

Each record has a tool identifier, a technician identifier, an issue time and a work bay, and the list is searched by tool identifier. Lists of 100, 1,000, 10,000 and 100,000 records were generated with seed 5067. For each size the number of comparisons was recorded for a search for the first record, the middle record, the last record, an identifier that is not on the list, and the mean of 1,000 random successful searches. The cost of one comparison was also timed in this environment, so that comparison counts could be turned into a response time.

n	First	Middle	Last	Not present	Mean of 1,000 successful	(n + 1)/2
100	1	51	100	100	51.5	50.5
1,000	1	501	1,000	1,000	507.7	500.5
10,000	1	5,001	10,000	10,000	5,048.1	5,000.5
100,000	1	50,001	100,000	100,000	51,353.6	50,000.5

Table 2.1: Comparisons of sequential search (from results/q2.csv)

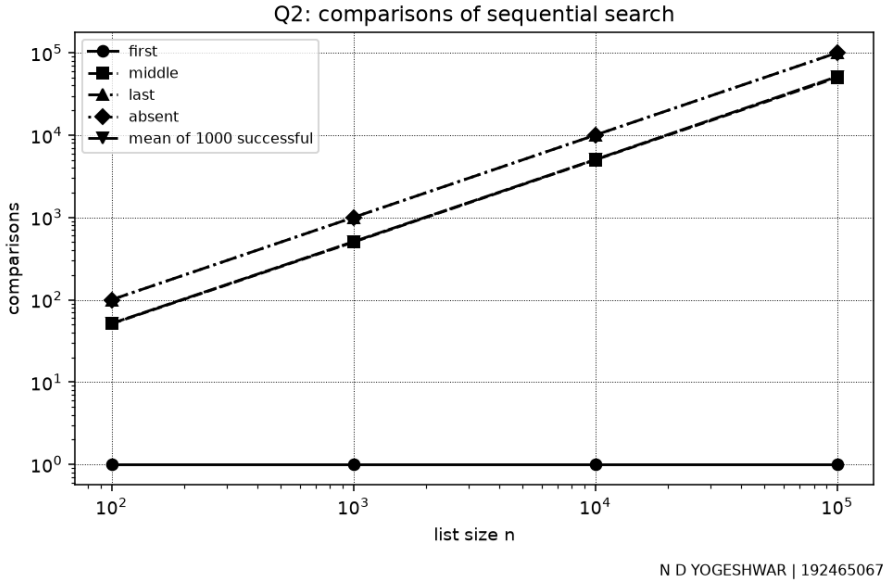


Figure 2.1: Comparisons against list size (logarithmic axes)

```
C:\DAA-A2\src>  python q2_sequential.py

================================================================================
Date/Time: 12-10-2026 22:27:38
Q2 - SEQUENTIAL SEARCH EXPERIMENTS
================================================================================
       n       first      middle        last      absent    mean (1000 hits)
     100           1          51         100         100                51.5
    1000           1         501        1000        1000               507.7
   10000           1        5001       10000       10000              5048.1
  100000           1       50001      100000      100000             51353.6

measured cost of one comparison in this environment: 128.1 ns
acceptable response time chosen: 1.0 ms -> unsuccessful search stays acceptable up to n = 7809 records

CHANGED CONDITION (assumed figures): one hangar vs 20 facilities
scenario                     records    requests/h   avg comps/req  comparisons per hour
one hangar                       400           300             300                 90000
20 facilities (one list)        8000          6000            6000             36000000
one unsuccessful search in the 20-facility list: 8000 comparisons = 1.02 ms

                                                    N D YOGESHWAR  |  Reg No 192465067
```

Figure 2.2: Program output for Q2

2.2 Observations

A successful search needs a number of comparisons equal to the position of the record, so the first record always costs one comparison and the last costs n. The mean over random successful searches

is very close to (n + 1)/2, as expected for a uniform choice of record. An unsuccessful search always costs n comparisons, because the whole list has to be examined before the system can say that the tool is not issued. This is the important asymmetry: a failed search is twice as expensive as the average successful one.

2.3 Operating boundary

One comparison cost about 128 ns in the run recorded here (the value varied between 87 and 124 ns in repeated runs on the same machine, and it includes the tuple access and string comparison of Python). I define an acceptable response time for a technician at the counter as 1 ms. An unsuccessful search stays inside that limit up to about 7,809 records in this run, and from the other runs the boundary lies between roughly 8,000 and 11,000 records. Below this size sequential search is a reasonable engineering choice, and it has real advantages for this application: the list changes all the time, a new record is added or removed in constant time, no sorted order or index has to be maintained, and the code is trivial to verify. Because tool identifiers are unique, a search can stop at the first match.

The second dimension is the frequency of requests. The comparisons per hour equal requests per hour multiplied by the average comparisons per request, so a list that is acceptable for one request may become a continuous load when requests arrive many times a second. I would stop recommending sequential search when either the list passes about 10,000 records or the request rate is high enough that the comparisons consume a noticeable share of the processor.

2.4 Changed condition: 20 maintenance facilities

The numbers of records and requests for a single hangar are not given in the assignment, so I assumed 400 active records and 300 requests per hour for one hangar, with half of the requests unsuccessful, and scaled both by 20 for the combined system. This gives 8000 records and 6000 requests per hour, an average of 6000 comparisons per request and 36,000,000 comparisons per hour, against 90,000 for one hangar.

Scenario	Records	Requests per hour	Average comparisons per request	Comparisons per hour
one hangar	400	300	300	90,000
20 facilities (one list)	8000	6000	6000	36,000,000

Table 2.2: Effect of expanding from one hangar to 20 facilities (assumed workload)

One unsuccessful search in the combined list needs 8000 comparisons, which is about 1.02 ms in this run, so the system sits right at my 1 ms limit. The total of 36 million comparisons per hour is still small for a processor, so sequential search would not fail technically. My revised recommendation is nevertheless to stop using it for the combined system: the list is at the edge of the response-time limit, any further growth crosses it, and 20 facilities would probably send requests concurrently. A hash table keyed by tool identifier gives expected constant-time lookup and also supports the continuous issue and return of tools in constant time, whereas a sorted array would make every update expensive. If the combined list were kept per facility (about 400 records each), sequential search would remain acceptable for each of the 20 lists.

Q3. Behaviour of Brute-Force String Matching

3.1 Test conditions and predictions

The matcher aligns the pattern at each of the n - m + 1 positions of the text and compares characters from left to right until a mismatch occurs or the whole pattern has matched. The work therefore depends on how many characters of the pattern match at each alignment before the first mismatch. Four test conditions were designed, each with n = 10,000 characters, and a prediction was written before running.

Test	Text and pattern	Prediction and reason
1 Little work	Text of the letter b only; pattern of ten letters a	The first character mismatches at every alignment, so one comparison per alignment: n - m + 1 = 9,991.
2 Engineered worst case	Text of the letter a only; pattern of nine a followed by b	Nine characters match and the tenth fails at every alignment: m(n - m + 1) = 99,910.
3 Own condition: log text	Random log lines (time, two words, number); pattern 'request timeout 10000', which cannot occur	Expected comparisons per alignment from the character frequencies (1 + p1 + p1p2 + ...); close to one, so about 10,300.
4 Own condition: four-letter alphabet	Random text over A, C, G, T; pattern of eleven letters that does not occur	Each letter matches with probability 1/4, so 1 + 1/4 + 1/16 + ... = 1.33 comparisons per alignment: about 13,300.

Table 3.1: Designed test conditions

3.2 Results

Test	n	m	Predicted comparisons	Observed comparisons
1 low work: no pattern letter in text	10000	10	9,991	9,991
2 engineered worst case	10000	10	99,910	99,910
3 log-style text and absent signature	10000	21	10,318	10,622
4 DNA-like alphabet (4 letters)	10000	11	13,304	13,254

Table 3.2: Predicted and observed character comparisons (from results/q3.csv)

Text length n (m = 10)	Predicted m(n - m + 1)	Observed
1,000	9,910	9,910
10,000	99,910	99,910
100,000	999,910	999,910

Table 3.3: Worst case for increasing text length

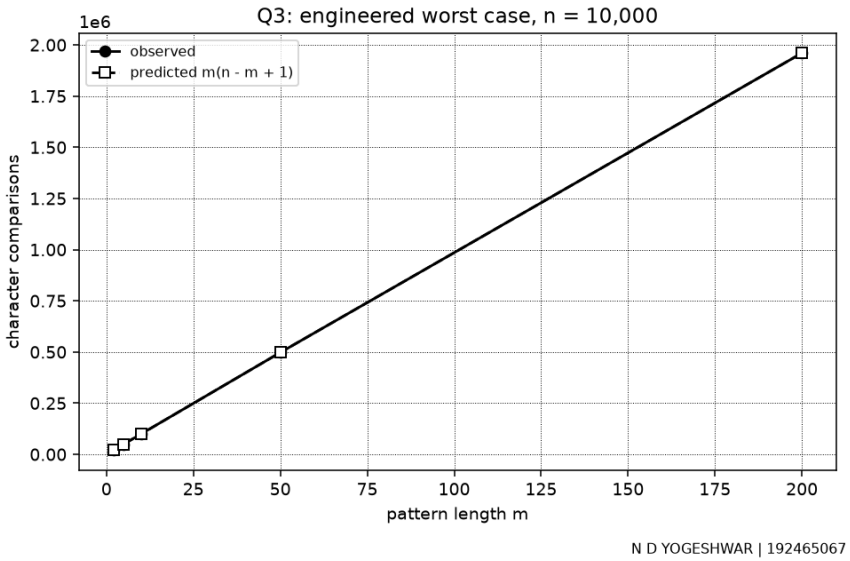


Figure 3.1: Engineered worst case against pattern length

```
C:\DAA-A2\src>  python q3_string_matching.py

========================================================================
Date/Time: 12-10-2026 22:27:47
Q3 - BRUTE-FORCE STRING MATCHING
========================================================================
PREDICTIONS (made before running)
  test 1: first character mismatches at every alignment -> n - m + 1 comparisons
  test 2: m - 1 characters match at every alignment, last one fails -> m (n - m + 1) comparisons
  test 3: character frequencies of the text give 1.034 comparisons per alignment -> about 10318
  test 4: each letter has probability 1/4 -> 1.332 per alignment -> about 13304

test                                        n     m   predicted    observed  found at
1 low work: no pattern letter in text      10000    10        9991        9991        -1
2 engineered worst case                    10000    10       99910       99910        -1
3 log-style text and absent signature      10000    21       10318       10622        -1
4 DNA-like alphabet (4 letters)            10000    11       13304       13254        -1

EFFECT OF TEXT LENGTH (engineered worst case, m = 10)
  n =   1000  observed     9910   predicted m(n-m+1) =     9910
  n =  10000  observed    99910   predicted m(n-m+1) =    99910
  n = 100000  observed   999910   predicted m(n-m+1) =   999910
EFFECT OF PATTERN LENGTH (engineered worst case, n = 10000)
  m =    2  observed    19998   predicted m(n-m+1) =    19998
  m =    5  observed    49980   predicted m(n-m+1) =    49980
  m =   10  observed    99910   predicted m(n-m+1) =    99910
  m =   50  observed   497550   predicted m(n-m+1) =   497550
  m =  200  observed  1960200   predicted m(n-m+1) =  1960200


                                                N D YOGESHWAR  |  Reg No 192465067
```

Figure 3.2: Program output for Q3

3.3 Inquiry

Test 1 and test 2 have exactly the same text length and pattern length, yet the second needed ten times as many comparisons. The difference comes from the content: in test 2 almost the entire pattern matches at every alignment before the mismatch at the last character, so each alignment repeats nine successful comparisons, and the text offers many overlapping partial matches. In test 1 the mismatch at the first character discards the alignment immediately. Repeated comparisons are

produced by a pattern whose prefix also occurs repeatedly in the text, by a small alphabet, and by a mismatch that occurs late in the pattern. The algorithm never uses what it learned at one alignment to skip the next one, and for this reason it re-reads the same text characters up to m times.

Predictions 1, 2 and 4 were confirmed almost exactly. Test 3 was observed about three percent above the prediction (10,622 against 10,318), because the prediction multiplies single-letter frequencies as if neighbouring letters were independent, whereas letters in the log lines are correlated; the prediction method is therefore good but not exact for natural text.

The result shows that the input size n (and m) is not sufficient to describe the running time of this algorithm. The cost of the three experiments with the same n lies between 9,991 and 99,910 comparisons, a factor of ten, and the worst case of m(n - m + 1) grows linearly in both parameters. An analysis in terms of input size alone must therefore be accompanied by the structure of the input, and it should state which case (best, average or worst) it describes.

Q4. Closest Pair and the Cost of Examining Every Possibility

4.1 Design and pair count

A brute-force solution has to look at the distance between every two different drones, because the closest pair could be any of them. A pair (i, j) is the same as the pair (j, i), and a drone is not paired with itself, so the loops run with i from 1 to n - 1 and j from i + 1 to n. The number of candidate pairs is therefore

```
sum over i = 1..n-1 of (n - i) = (n-1) + (n-2) + ... + 1 = n(n - 1) / 2
```

For 100 drones this is 4,950 pairs, and doubling n multiplies the number of pairs by about four. Squared distances ($dx^2 + dy^2$) are compared instead of distances, because the square root is monotonic, so the closest pair is the same, and one square root per pair is avoided. Each examined pair requires exactly one distance calculation, so the number of distance calculations equals the number of pairs.

```
best = infinity
for i = 0 to n-2
    for j = i+1 to n-1
        d = (xi - xj)^2 + (yi - yj)^2
        if d < best: best = d; pair = (i, j)
return pair
```

4.2 Results

n	n(n - 1)/2	Pairs examined	Distance calculations	Time (s)
100	4,950	4,950	4,950	0.0023
200	19,900	19,900	19,900	0.0071
400	79,800	79,800	79,800	0.0281
800	319,600	319,600	319,600	0.1053
1,600	1,279,200	1,279,200	1,279,200	0.4911
3,200	5,118,400	5,118,400	5,118,400	1.8432
5,000	12,497,500	12,497,500	12,497,500	4.6532

Table 4.1: Closest-pair experiments (from results/q4.csv)

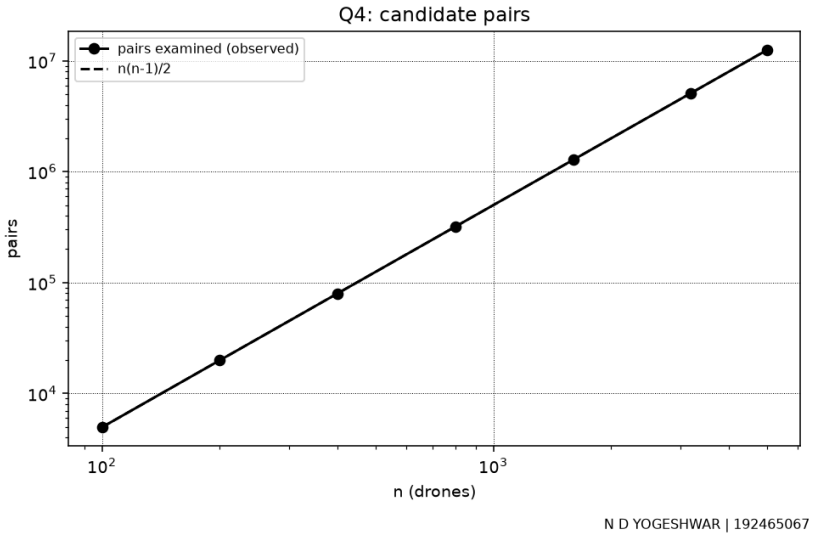


Figure 4.1: Pairs examined and the theoretical n(n - 1)/2

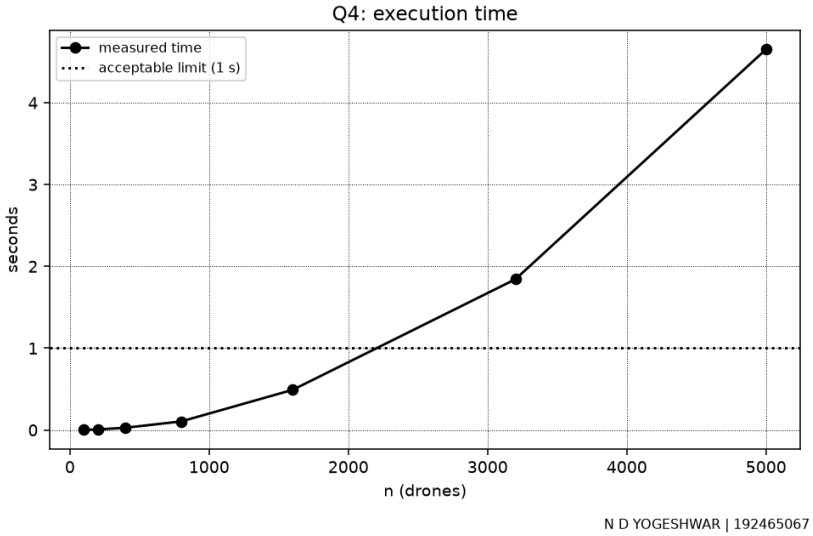


Figure 4.2: Execution time against n

```
C:\DAA-A2\src>  python q4_closest_pair.py

================================================================================
Date/Time: 12-10-2026 22:27:48
Q4 - BRUTE-FORCE CLOSEST PAIR
================================================================================
       n          n(n-1)/2    pairs examined    distance calcs    time (s)
     100              4950              4950              4950      0.0023
     200             19900             19900             19900      0.0071
     400             79800             79800             79800      0.0281
     800            319600            319600            319600      0.1053
    1600           1279200           1279200           1279200      0.4911
    3200           5118400           5118400           5118400      1.8432
    5000          12497500          12497500          12497500      4.6532

measured cost of one pair: 372 ns
acceptable response time chosen: 1.0 s -> brute force stays acceptable up to about n = 2318 drones
  estimated time for n =  10000 :      18.6 s
  estimated time for n =  50000 :     465.4 s
  estimated time for n = 100000 :    1861.6 s

                                                  N D YOGESHWAR  |  Reg No 192465067
```

Figure 4.3: Program output for Q4

The observed number of pairs is identical to n(n - 1)/2 for all seven sizes, so the theoretical derivation matches the implementation exactly. When n doubled from 1,600 to 3,200 the pair count grew by a factor of 4.0 and the time by about 4, which is consistent with quadratic growth.

4.3 Scalability

For this experimental environment I take an acceptable response time to be one second, which is a reasonable limit for a monitoring display that is refreshed every few seconds. One pair cost about 372 ns in the largest run, so the time t(n) = 372 ns x n(n - 1)/2 reaches one second at about n = 2,318 drones. This agrees with the table, where 1,600 drones took 0.49 s and 3,200 drones took 1.84 s. Extrapolating, 10,000 drones would need roughly 19 s and 100,000 drones roughly 31 minutes. Brute force becomes impractical for this environment at a few thousand drones.

The inefficiency comes from the fact that every pair is treated as a separate, independent question: the algorithm does not use the positions of the points to rule out pairs that are obviously far apart. Most of the n(n - 1)/2 distances can never be the minimum, but they are still calculated. A redesigned strategy would have to avoid comparing points that are too far from each other to beat the best distance found so far, for example by ordering or partitioning the points in space so that each point is compared only with its neighbourhood, and by reusing the current best distance as a pruning limit.

Q5. A Convex Hull Algorithm from a Geometric Principle

5.1 Derivation of the decision rule

Two different points $p_i = (x_i, y_i)$ and p_j define a line. The position of a third point p_k relative to the directed line from p_i to p_j is given by the sign of the cross product of the vectors p_ip_j and p_ip_k:

```
s = (xj - xi)(yk - yi) - (yj - yi)(xk - xi)
s > 0 : pk lies to the left of the line
s < 0 : pk lies to the right of the line
s = 0 : pk lies on the line
```

The principle given in the assignment says that the segment p_ip_j can be part of the boundary only if all the other points lie on one side of the line. In terms of the sign, the segment is a hull edge if no pair of points k and l exists with $s_k > 0$ and $s_l < 0$. A point with $s = 0$ needs separate treatment: if it lies between p_i and p_j it does no harm, but if it lies outside the segment, then p_ip_j is only part of a longer boundary edge and must be rejected, because the longer segment between the two extreme points will be accepted instead. Duplicate points are removed first.

```
remove duplicate points
for every pair (i, j), i < j
    seenLeft = seenRight = false ; ok = true
    for every other point k
        s = cross(pi, pj, pk)
        if s > 0: seenLeft = true
        else if s < 0: seenRight = true
        else if pk is not between pi and pj: ok = false ; stop
        if seenLeft and seenRight: ok = false ; stop
    if ok: accept edge (i, j)
hull vertices = end points of accepted edges
order the vertices by angle around their centroid (only for drawing)
```

5.2 Test configurations

Set A has six points on a hexagon-like outer ring and thirty interior points, so it contains clear boundary and interior points. Set B was designed to challenge the rule: the four corners of a square, points in the middle of the sides and other points on the sides (collinear with the corners), a duplicated corner, interior grid points, and points very close to the boundary such as (19, 19) and (10, 1). Set C has eight collinear points, for which the hull degenerates into a segment.

Set	Points given	Distinct points	Orientation tests	Hull vertices
A	36	36	6,364	6
B	23	21	1,395	4

Table 5.1: Results for the two designed sets (from results/q5set.csv)

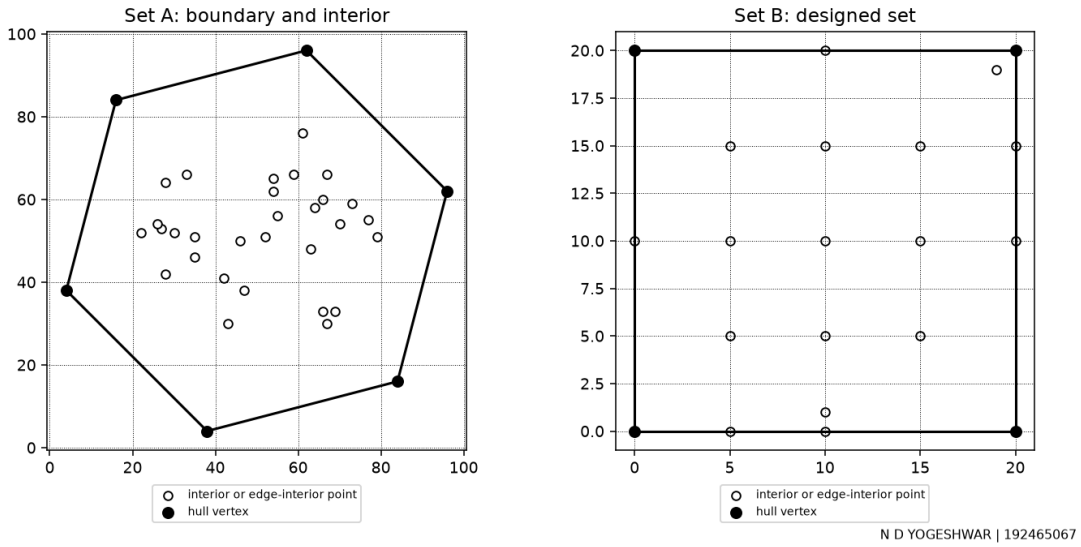


Figure 5.1: Boundary found for set A (left) and set B (right)

```
C:\DAA-A2\src>  python q5_convex_hull.py

========================================================================
Date/Time: 12-10-2026 22:27:55
Q5 - BRUTE-FORCE CONVEX HULL
========================================================================
Set A: boundary and interior points
  points given: 36, distinct: 36, orientation tests: 6364, accepted edges: 6
  hull vertices (counter-clockwise): [(4, 38), (38, 4), (84, 16), (96, 62), (62, 96), (16, 84)]
Set B: designed (collinear, duplicates)
  points given: 23, distinct: 21, orientation tests: 1395, accepted edges: 4
  hull vertices (counter-clockwise): [(0, 0), (20, 0), (20, 20), (0, 20)]
Set C: all points collinear -> hull vertices: [(0, 0), (7, 14)] (tests 48)

       n       pairs    tests (random)    tests (circle) hull (rand)    time (s)
      10          45               185               227           6      0.0002
      20         190              1388              1652           7      0.0010
      40         780              8972             12102           7      0.0062
      80        3160             57493             91402          11      0.0379
     160       12720            468404            707602          11      0.2342
     320       51040           3236479           5562402          14      1.9039

                                                  N D YOGESHWAR | Reg No 192465067
```

Figure 5.2: Program output for Q5

For set B the algorithm returned exactly the four corners; the points at (10, 0), (20, 10), (5, 0) and (20, 15), which lie on the boundary but between corners, were correctly not reported as vertices, and the duplicate corner was reported once. For set C the result was the two end points of the line.

5.3 Growth of the work

n	Pairs	Tests, random points	Tests, points on a circle	Hull vertices (random)	Time (s)
10	45	185	227	6	0.0002
20	190	1,388	1,652	7	0.0010
40	780	8,972	12,102	7	0.0062
80	3,160	57,493	91,402	11	0.0379
160	12,720	468,404	707,602	11	0.2342
320	51,040	3,236,479	5,562,402	14	1.9040

Table 5.2: Orientation tests as the number of points grows (from results/q5.csv)

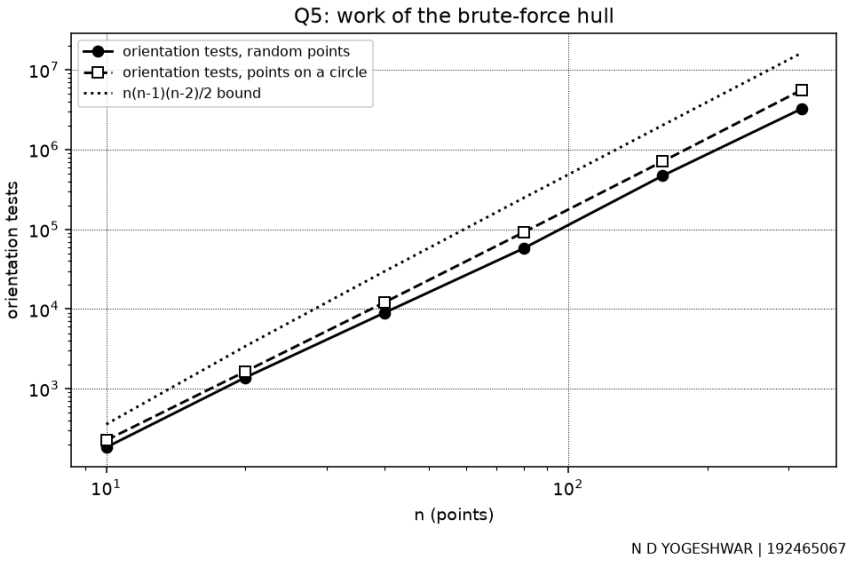


Figure 5.3: Orientation tests against n (logarithmic axes)

There are n(n - 1)/2 pairs, and each pair is tested against at most n - 2 other points, so the worst case is n(n - 1)(n - 2)/2 orientation tests, which is $O(n^3)$. The measured counts grow by a factor of about 6.4 to 8 when n doubles, close to the factor of 8 of a cubic function. The counts are below the bound because a pair that is not a hull edge is usually rejected as soon as one point is found on each side. For 320 points the bound is about 16.2 million tests, and 3.2 million (random points) and 5.6 million (points on a circle) were needed. Points that are all on the hull give more tests, because the rejection of a pair takes longer to occur.

5.4 Why an accepted edge belongs to the boundary

The convex hull of a set of points is the intersection of all half-planes that contain the whole set. If a line through p_i and p_j has every other point on one closed side of it, then that closed half-plane contains the whole set, and so it is one of the half-planes whose intersection forms the hull. The line therefore touches the hull, because it passes through p_i and p_j, which are points of the hull. The segment lies on the line, and it also lies inside the hull, as both of its end points are in the hull and the hull is convex. A segment that lies in the hull and on a supporting line must lie on the boundary, since an interior point of the hull has a small disc around it inside the hull, which would cross the supporting line. This is why one-sidedness is sufficient. It is also necessary: if points were on both sides, the line would pass through the interior of the hull. Collinear points need the extra rule because a supporting line may contain several points, and only the two extreme ones are vertices.

Q6. Exhaustive Search and Combinatorial Growth

6.1 Design of the enumeration

A candidate solution is a subset of the n packages. A subset is represented as an n-bit number, in which bit i is 1 when package i is loaded. All candidates are generated by counting the integers from 0 to 2^n - 1. For each candidate the weights and values of the chosen packages are added; the candidate is feasible if the total weight does not exceed the payload limit, a feasible candidate is evaluated by its total value, and the best value and its bit pattern are retained whenever a feasible candidate improves them.

```
best = -1
```

```
for mask = 0 to 2^n - 1
    w = 0 ; v = 0
    for i = 0 to n-1
        if bit i of mask is 1: w = w + weight[i] ; v = v + value[i]
    if w <= capacity and v > best: best = v ; bestMask = mask
return best, bestMask
```

6.2 Correctness check on a small instance

A small instance with five packages and a payload limit of 10 kg was created, so that all $2^5 = 32$ subsets can be written down and checked by hand. The packages are P1 (7 kg, value 42), P2 (3 kg, 12), P3 (4 kg, 40), P4 (5 kg, 25) and P5 (2 kg, 10). The feasible subsets are listed below; the other 16 exceed 10 kg.

Subset	Weight	Value
{}	0	0
{P1}	7	42
{P2}	3	12
{P3}	4	40
{P4}	5	25
{P5}	2	10
{P1,P2}	10	54
{P1,P5}	9	52
{P2,P3}	7	52
{P2,P4}	8	37
{P2,P5}	5	22
{P3,P4}	9	65
{P3,P5}	6	50
{P4,P5}	7	35
{P2,P3,P5}	9	62
{P2,P4,P5}	10	47

Table 6.1: Feasible subsets of the verification instance

By inspection the best feasible subset is {P3,P4} with weight 9 kg and value 65. The exhaustive search returned exactly this subset and the same value, and an independent check that uses `itertools.combinations` gave the same result (output in Figure 6.1).

```
C:\DAA-A2\src>  python q6_exhaustive.py

========================================================================
Date/Time: 12-10-2026 22:28:01
Q6 - EXHAUSTIVE SEARCH (HELICOPTER PAYLOAD)
========================================================================
Verification instance, capacity 10 kg
  P1 weight 7 value 42
  P2 weight 3 value 12
  P3 weight 4 value 40
  P4 weight 5 value 25
  P5 weight 2 value 10
all 32 subsets:
  subset                  weight   value  feasible
  {}                           0       0       yes
  {P1}                         7      42       yes
  {P2}                         3      12       yes
  {P1,P2}                     10      54       yes
  {P3}                         4      40       yes
  {P1,P3}                     11      82        no
  {P2,P3}                      7      52       yes
  {P1,P2,P3}                  14      94        no
  {P4}                         5      25       yes
  {P1,P4}                     12      67        no
  {P2,P4}                      8      37       yes
  {P1,P2,P4}                  15      79        no
  {P3,P4}                      9      65       yes
  {P1,P3,P4}                  16     107        no
  {P2,P3,P4}                  12      77        no
  {P1,P2,P3,P4}               19     119        no
  {P5}                         2      10       yes
  {P1,P5}                      9      52       yes
  {P2,P5}                      5      22       yes
  {P1,P2,P5}                  12      64        no
  {P3,P5}                      6      50       yes
  {P1,P3,P5}                  13      92        no
  {P2,P3,P5}                   9      62       yes
  {P1,P2,P3,P5}               16     104        no
  {P4,P5}                      7      35       yes
  {P1,P4,P5}                  14      77        no
  {P2,P4,P5}                  10      47       yes
  {P1,P2,P4,P5}               17      89        no
  {P3,P4,P5}                  11      75        no
  {P1,P3,P4,P5}               18     117        no
  {P2,P3,P4,P5}               14      87        no
  {P1,P2,P3,P4,P5}            21     129        no
exhaustive search: best value 65 with ['P3', 'P4'], feasible subsets 16
independent check (itertools.combinations): best value 65 -> MATCH

GROWTH (capacity = 40 percent of total weight)
    n           2^n    candidates      feasible  best value    time (s)
   10          1024          1024           298         400      0.0022
   12          4096          4096          1132         390      0.0121
   14         16384         16384          4027         433      0.0459
   16         65536         65536         16798         723      0.2283
   18        262144        262144         60566         673      0.7099
   20       1048576       1048576        223607         681      3.2712
   21       2097152       2097152        441304         707      6.6303

time per candidate: 3.16 microseconds
response time chosen for the planner: 5 s -> largest deployable n about 20 packages
  predicted time for n = 25 : 106.1 s
  predicted time for n = 30 : 3394.7 s
  predicted time for n = 40 : 3476204.5 s
  predicted time for n = 50 : 112.9 years

                                                  N D YOGESHWAR | Reg No 192465067
```

Figure 6.1: Program output for Q6

6.3 Growth: prediction and experiment

The number of subsets of n packages is 2^n, because each package is independently either in or out. The prediction made before the experiment was therefore that the number of candidates doubles for every additional package, and that the execution time, which is proportional to the number of

candidates (each needs an inner loop of n steps), roughly doubles as well.

n	2^n	Candidates examined	Feasible	Best value	Time (s)	Time ratio to previous n
10	1,024	1,024	298	400	0.0022	-
12	4,096	4,096	1,132	390	0.0121	5.50
14	16,384	16,384	4,027	433	0.0459	3.79
16	65,536	65,536	16,798	723	0.2283	4.97
18	262,144	262,144	60,566	673	0.7099	3.11
20	1,048,576	1,048,576	223,607	681	3.2712	4.61
21	2,097,152	2,097,152	441,304	707	6.6303	2.03

Table 6.2: Exhaustive search for increasing n, capacity 40 percent of the total weight (from results/q6.csv)

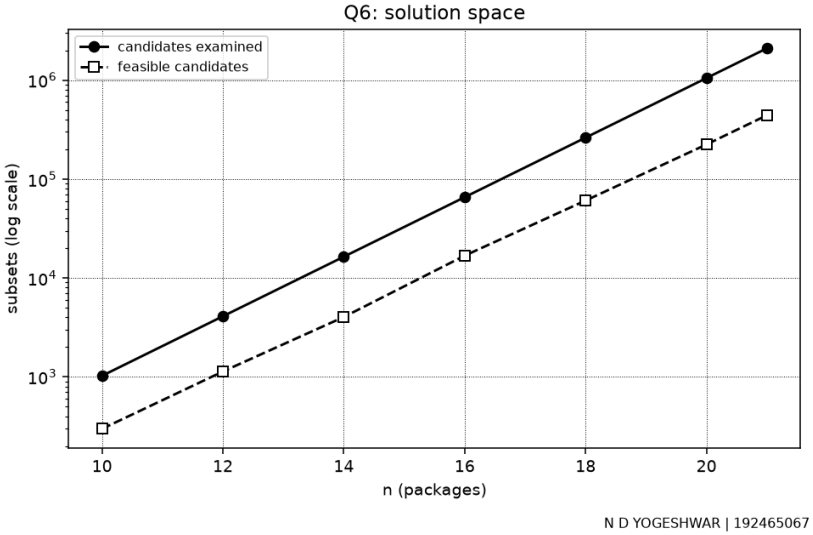


Figure 6.2: Candidate subsets against n (logarithmic axis)

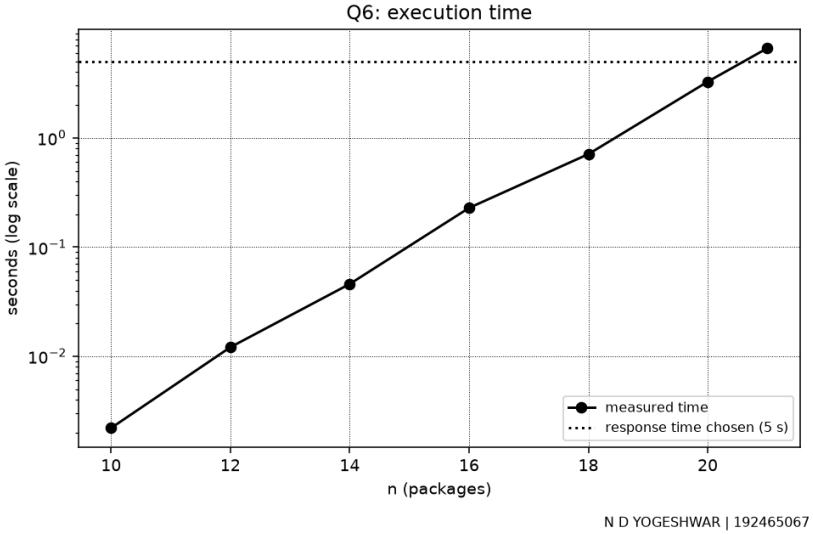


Figure 6.3: Execution time against n (logarithmic axis)

The number of candidates examined was exactly 2^n for every size, so the prediction for the solution space was confirmed. The ratio of the times for n = 20 and n = 21 is about two, as predicted. The

time ratios for two additional packages (n = 14 to 16, 16 to 18, 18 to 20) are about 4 to 5, again consistent with $2^2 = 4$ plus the extra inner-loop work.

6.4 Why a few more packages are expensive

Every new package doubles the number of candidate subsets, because each of the existing subsets can be taken with or without it. Adding k packages therefore multiplies the work by 2^k: ten more packages mean 1,024 times more candidates, and twenty more mean more than a million times. The increase in the size of the input is additive, but the solution space is multiplicative, and the algorithm examines the entire solution space.

6.5 Deployment decision

For the airlift planner I select a response time of 5 seconds, since a loading decision that is requested interactively should appear while the operator is still looking at the screen. A candidate cost about 3.2 microseconds in this environment, so n = 21 packages needed 6.6 s. I would therefore deploy this exhaustive search only up to about 20 packages in this environment. At 25 packages the same program would need about 106 seconds, at 30 packages about 57 minutes and at 40 packages about 40 days. A compiled language would shift this limit by only a few packages, since a speed-up of one hundred times is worth only about seven additional packages.

Q7. Day-4 Design Challenge: Diagnose, Redesign and Validate

The solution selected is the exhaustive search of Q6, because it is the only one of my solutions for which the experiments show a limit that cannot be removed by waiting for a faster machine: its effort doubles with every package.

A. Diagnosis of the bottleneck

Almost all of the effort is spent in the double loop of the enumeration: 2^n candidates are generated, and for each of them the inner loop makes n steps, so about $n\,2^n$ additions are made. The operation that grows is therefore the generation and evaluation of a candidate, and in Table 6.2 the feasible candidates are only a quarter or less of the total, which means that most of the work is spent on subsets that are too heavy. For n = 20 only 223,607 of 1,048,576 candidates were feasible.

Work is also repeated. The weight and value of a subset are recalculated from zero although two subsets that differ only in the last package share all the other additions. More importantly, once a set of packages is already over the payload limit, every superset of it is also over the limit, but the enumeration still builds and tests all of them separately (a heavy set of k packages has 2^{n-k} supersets). The cause of the limitation is therefore a property of the original algorithm: it treats the candidates as unrelated and independent numbers, instead of as the leaves of a decision tree in which a decision on one package restricts all of the later ones.

B. Design objective

The redesign must (1) avoid generating subsets that cannot be feasible or cannot beat the best load found so far, (2) reuse the partial weight and value instead of recomputing them, and (3) still return the exact optimum. The measurable objective is that, for random instances with n = 20, the number of search nodes visited should be at most 1 percent of the 2^n candidates of the original, with the same best value on every instance that both programs can solve.

C. The revised strategy

The candidates are organised as a decision tree. At depth i the algorithm decides whether to take package i, and it carries the weight and value of the decisions made so far, so that each node costs O(1) instead of O(n). Three changes are made. First, a branch that takes a package which would exceed the limit is not generated, so the whole subtree of overweight supersets disappears. Second, the packages are sorted in decreasing order of value per kg before the search, so the first complete loads found are good ones and the best value rises quickly. Third, a bound is calculated at every node: the value already collected plus the most that the remaining packages could add. The remaining packages can add at most their total value, and also at most the free capacity multiplied by the best density still available (the density of the next package, since they are sorted), so the smaller of the two is used. If this bound is not greater than the best value already found, the subtree cannot improve the solution and it is skipped.

```
sort packages by value/weight, largest first
suffix[i] = total value of packages i..n-1
best = 0
search(i, w, v):
    if v > best: best = v
    if i == n: return
    bound = v + min(suffix[i], (capacity - w) * value[i] / weight[i])
    if bound <= best: return                      // prune: cannot improve
    search(i + 1, w, v)                           // leave package i
    if w + weight[i] <= capacity:                 // take package i only if feasible
        search(i + 1, w + weight[i], v + value[i])
```

The redesign introduces preprocessing of O(n log n) for the sort and O(n) memory for the suffix sums and the stack (the implementation uses an explicit stack). The new trade-off is that the running time is no longer fixed by n: it depends on the instance, and the worst case remains exponential.

D. Prediction before implementation

I predicted that, on random instances, the number of nodes visited would be a very small fraction of 2^n and would not double with every package; that the saving would be smaller on strongly correlated instances (value equal to weight plus 10), because the density of all packages is then almost the same and the bound is weak; and that on instances where all packages have exactly the same density and the capacity cannot be filled exactly, the bound could never cut anything, so the advantage would almost vanish. The performance measure expected to improve is the number of candidates (nodes) examined.

E. Implementation and comparison

Instance type: random

n	Original: 2^n candidates	Feasibility pruning only (nodes)	Feasibility and bound (nodes)	Best value	Time of original (s)	Time of redesign (s)
12	4,096	3,477	87	463	0.0093	0.00007
16	65,536	44,361	198	550	0.1750	0.00013
20	1,048,576	644,449	208	766	3.0022	0.00028
22	4,194,304	2,588,038	501	847	13.4704	0.00031
24	16,777,216	9,262,485	200	780	-	0.00020

n	Original: 2^n candidates	Feasibility pruning only (nodes)	Feasibility and bound (nodes)	Best value	Time of original (s)	Time of redesign (s)
28	268,435,456	-	1,308	1064	-	0.00129
32	4,294,967,296	-	4,633	1236	-	0.00258
36	68,719,476,736	-	23,586	1470	-	0.01618
40	1,099,511,627,776	-	16,418	1444	-	0.02022

Instance type: correlated

n	Original: 2^n candidates	Feasibility pruning only (nodes)	Feasibility and bound (nodes)	Best value	Time of original (s)	Time of redesign (s)
12	4,096	3,366	259	199	0.0054	0.00013
16	65,536	49,442	625	275	0.1556	0.00036
20	1,048,576	688,058	2,209	341	3.0552	0.00199
22	4,194,304	2,614,044	881	372	14.9534	0.00086
24	16,777,216	10,546,600	3,072	384	-	0.00276
28	268,435,456	-	13,686	479	-	0.01101
32	4,294,967,296	-	3,627	546	-	0.00324
36	68,719,476,736	-	78,683	602	-	0.06177
40	1,099,511,627,776	-	110,185	664	-	0.08531

Instance type: equal density and odd capacity

n	Original: 2^n candidates	Feasibility pruning only (nodes)	Feasibility and bound (nodes)	Best value	Time of original (s)	Time of redesign (s)
12	4,096	3,029	2,189	144	0.0059	0.00117
16	65,536	37,350	16,958	188	0.1491	0.01573
20	1,048,576	587,128	313,142	218	3.3082	0.26654
22	4,194,304	1,967,644	878,770	286	14.7555	0.72080
24	16,777,216	7,881,821	4,004,883	260	-	3.32951

Table 7.1: Original and redesigned search (from results/q7.csv). The original was run only up to n = 22; for larger n the number 2^n is shown. The best values of both programs were identical on every instance that both could solve.

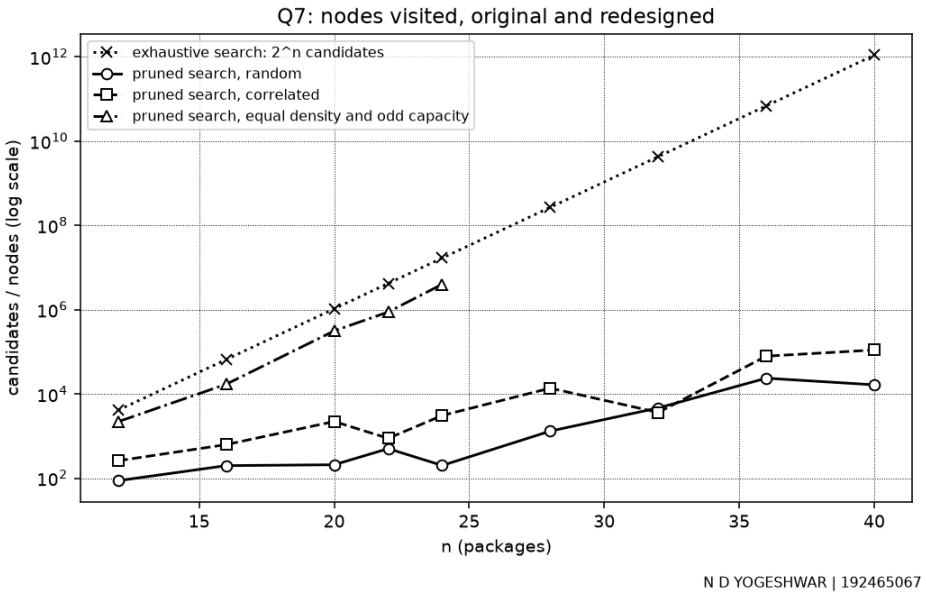


Figure 7.1: Candidates of the original and nodes of the redesign

```
C:\DAA-A2\src>  python q7_redesign.py

========================================================================
Date/Time: 12-10-2026 22:28:12
Q7 - REDESIGN OF THE EXHAUSTIVE SEARCH (PRUNED SEARCH)
========================================================================
PREDICTION (made before running): the number of nodes visited will be a very small fraction of 2^n
on random instances, and the saving will shrink on strongly correlated instances where value is
almost proportional to weight. If every item has the same value density and the capacity cannot be
filled exactly, the bound should never be able to cut anything, and the search should lose its advantage.

Instance type: random (capacity = 40 percent of total weight)
    n    exhaustive   feasibility    feas+bound  best value  t exh (s)   t pruned   same
   12          4096          3477            87         463      0.009     0.0001   True
   16         65536         44361           198         550      0.175     0.0001   True
   20       1048576        644449           208         766      3.002     0.0003   True
   22       4194304       2588038           501         847     13.470     0.0003   True
   24      16777216       9262485           200         780          -     0.0002      -
   28     268435456             -          1308        1064          -     0.0013      -
   32    4294967296             -          4633        1236          -     0.0026      -
   36   68719476736             -         23586        1470          -     0.0162      -
   40 1099511627776             -         16418        1444          -     0.0202      -

Instance type: correlated (capacity = 40 percent of total weight)
    n    exhaustive   feasibility    feas+bound  best value  t exh (s)   t pruned   same
   12          4096          3366           259         199      0.005     0.0001   True
   16         65536         49442           625         275      0.156     0.0004   True
   20       1048576        688058          2209         341      3.055     0.0020   True
   22       4194304       2614044           881         372     14.953     0.0009   True
   24      16777216      10546600          3072         384          -     0.0028      -
   28     268435456             -         13686         479          -     0.0110      -
   32    4294967296             -          3627         546          -     0.0032      -
   36   68719476736             -         78683         602          -     0.0618      -
   40 1099511627776             -        110185         664          -     0.0853      -

Instance type: equal density and odd capacity (capacity = 40 percent of total weight)
    n    exhaustive   feasibility    feas+bound  best value  t exh (s)   t pruned   same
   12          4096          3029          2189         144      0.006     0.0012   True
   16         65536         37350         16958         188      0.149     0.0157   True
   20       1048576        587128        313142         218      3.308     0.2665   True
   22       4194304       1967644        878770         286     14.756     0.7208   True
   24      16777216       7881821       4004883         260          -     3.3295      -

                                                                    N D YOGESHWAR | Reg No 192465067
```

Figure 7.2: Program output for Q7

F. Evaluation

The first prediction was supported. For random instances with n = 20 the redesign visited 208 nodes against 1,048,576 candidates, which is 0.020 percent, so the objective of 1 percent was met by a large margin, and with n = 40 it visited only 16,418 nodes, where the original would have needed 1,099,511,627,776 candidates (about 10^{12}), which is out of reach for it. Feasibility pruning alone reduced the work only to about 60 percent of 2^n (644,449 nodes at n = 20), so almost all of the improvement comes from the bound and from the density order.

The second prediction was supported in direction but not in size. The correlated instance with n = 20 needed 2,209 nodes, about ten times more than the random one, yet still only 0.21 percent of 2^n; I had expected a weaker saving. The third prediction was clearly supported: on the equal-density instance with an odd capacity, n = 24 needed 4,004,883 nodes against 16,777,216 candidates, a saving of only about 76 percent, which is just the effect of the feasibility rule. The node count does not increase smoothly with n in the tables, because every row is a new random instance, so the counts depend on the instance, not only on n. This is the main new cost of the redesign: its running

time cannot be predicted from n alone.

The redesign therefore addresses the limitation for typical instances: the planner could handle 40 packages in a fraction of a second where the original is unusable beyond about 20. It does not remove the exponential worst case, and no guarantee can be given for adversarial instances. The original exhaustive search is still preferable in three situations: when n is small (about 12 or less, where it takes milliseconds and is trivially correct), when a fixed and predictable response time is more important than speed on average, and as an independent reference used to verify the optimiser, which is how it was used here. The extra costs of the redesign are the sorting step, the memory for the stack and the more complicated code, whose correctness depends on the bound being valid; this was checked by comparing the best values with the original for every size up to 22.

Conclusion

The experiments in this assignment support four general conclusions. First, the best algorithm among several simple ones depends on the input: insertion sort is the right choice for nearly sorted orders and selection sort is never preferable unless swaps are expensive. Second, input size alone does not decide running time, as the string matching tests with identical n and m differed by a factor of ten. Third, the brute-force solutions of closest pair, convex hull and exhaustive search have polynomial or exponential growth that is predicted exactly by counting, and their practical limits were estimated from the measurements: a few thousand points for closest pair, a few hundred for the cubic hull, and about 20 packages for the exhaustive search at a five-second response time. Fourth, a redesign that exploits the structure of the problem, such as pruning by feasibility and bound, can bring an exponential problem within reach for typical instances, but only an analysis of instances where it fails shows what the improvement really guarantees.

References

T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 3rd edition, MIT Press, 2009.

A. Levitin, Introduction to the Design and Analysis of Algorithms, 3rd edition, Pearson, 2012.

D. E. Knuth, The Art of Computer Programming, Volume 3: Sorting and Searching, 2nd edition, Addison-Wesley, 1998.